

Informatika 3

Prednáška 8: Šablóny tried a funkcií



Generické programovanie



Generické programovanie

- Šablóny jazyka C++ sú jedným zo spôsobov realizácie programátorskej techniky generického programovania
- Objavilo sa v jazykoch ML (1973) a Ada (1977)
- Jednými z priekopníkov sú Alexander Stepanov a David Musser, neskôr Andrei Alexandrescu
- Hlavnou myšlienkou generického programovania je *abstrahovanie* od konkrétnych typov pri písaní algoritmov
- Neoceniteľné uplatnenie má pri tvorbe knižníc

Generické programovanie

- Generické algoritmy sú parametrizované nielen údajmi, s ktorými pracujú, ale aj ich typom – majú **typové parametre**
- Algoritmus môže od typu požadovať určité vlastnosti
- Takéto obmedzenie vo všeobecnosti nazývame koncept

```
max :: (Ord a) => a -> a -> a
max x y = if x < y then y else x
```

Funkcia `max` v jazyku [Haskell](#), ktorá vráti maximum z dvoch hodnôt `x` a `y` typu `a` (typový parameter). Hodnoty typu `a` musí byť možné porovnávať relačnými operátormi.

```
contains :: (Eq a) => a -> [a] -> Bool
contains _ [] = False
contains y (x:xs) = if x == y then True else contains y xs
```

Funkcia `contains` v jazyku [Haskell](#), ktorá zistí, či sa v zozname `xs` hodnôt typu `a` nachádza hodnota `y`. Hodnoty typu `a` musí byť možné porovnávať na rovnosť a nerovnosť.

Podpora generického programovania

- Väčšina moderných programovacích jazykov podporuje generické programovanie
- Typickým príkladom je implementácia kontajnerov poskytovaných štandardnou knižnicou väčšiny programovacích jazykov
- Podpora sa medzi jazykmi líši od veľmi jednoduchej (Java) po sofistikovanú (C++, Haskell)
- Generické programovanie nie je viazané na objektové programovanie
- Jazyky sa líšia technickou realizáciou generického kódu a konceptov a mierou možnosti práce s typovými parametrami

Jazyk Java

- V jazyku Java je generické programovanie realizované formou generík
- Využíva techniku zvanú type erasure
- Využíva skutočnosť, že všetky triedy sú v potomkom triedy `Object`

```
class Optional<T> {  
    private T value;  
    private boolean hasValue;  
  
    public Optional() {  
        this.value = null;  
        this.hasValue = false;  
    }  
  
    public Optional(T value) {  
        this.value = value;  
        this.hasValue = true;  
    }  
  
    public T getValue() {  
        return this.value;  
    }  
  
    public boolean hasValue() {  
        return this.hasValue;  
    }  
};
```

Jazyk Java

- V programe tak existuje len jedna *generická* trieda, v ktorej sa všetky výskyty typových parametrov nahradia typom `Object`
- Na miesta použitia metód pridá kompilátor pretypovanie
- Veľkou nevýhodou je nemožnosť použitia primitívnych typov ako hodnôt typových parametrov
- Java tento problém rieši tzv. autoboxingom

```
class Optional {  
    private Object value;  
    private boolean hasValue;  
  
    public Optional() {  
        this.value = null;  
        this.hasValue = false;  
    }  
  
    public Optional(Object value) {  
        this.value = value;  
        this.hasValue = true;  
    }  
  
    public Object getValue() {  
        return this.value;  
    }  
  
    public boolean hasValue() {  
        return this.hasValue;  
    }  
};
```

Jazyk Java

```
public static void main(String[] args) {  
    Optional<Integer> opti = new Optional<Integer>(10);  
    Optional<Double> optd = new Optional<Double>(1.41);  
    if (opti.hasValue()) {  
        System.out.println(opti.getValue());  
    }  
    if (optd.hasValue()) {  
        System.out.println(optd.getValue());  
    }  
}
```

```
public static void main(String[] args) {  
    Optional opti = new Optional(10);  
    Optional optd = new Optional(1.41);  
    if (opti.hasValue()) {  
        System.out.println((Integer)opti.getValue());  
    }  
    if (optd.hasValue()) {  
        System.out.println((Double)optd.getValue());  
    }  
}
```

Typová kontrola a pridanie pretypovaní prebehne pri kompilácii. Po nej sa typ *T stratí* (je vymazaný) – angl. **type erasure**.

Jazyk Java umožňuje špecifikovať vlastnosti typového parametra (koncept) prostredníctvom angl. wildcards.

Šablóny v jazyku C++



Šablóny v jazyku C++

- V jazyku C++ používame na realizáciu generického programovania mechanizmus zvaný šablóna
- Šablóny sú jedným z najvýznamnejších prvkov jazyka C++
- Šablóna parametrizuje časť kódu (trieda, funkcia, premenná, alias) typovými parametrami
- **Kompilátor** zabezpečí vytvorenie **inštancie šablóny** pre každú použitú kombináciu hodnôt typových parametrov
- Rôzne inštancie šablóny sú rozdielne typy

Šablóny v jazyku C++

- Definíciu šablóny začíname kľúčovým slovom `template` nasledovaným párom `<>` obsahujúcim zoznam **typových parametrov**
- Typový parameter uvádzame kľúčovým slovom `typename` a jeho **identifikátorom**
- Za nimi nasleduje štandardná definícia danej entity, v ktorej používame typové parametre ako typy

```
template<typename T>
class Optional {
public:
    Optional();
    Optional(T value);
    bool hasValue() const;
    T &getValue();
    const T &getValue() const;

private:
    T m_value;
    bool m_hasValue;
};
```

Definície členov

- Pri definícii členov šablóny (ak ide o šablónu triedy) musíme opakovať uvedenie šablóny
- Pri uvádzaní vlastníka metódy (šablóny) triedy uvádzame aj zoznam typových parametrov

Optional<T>::hasValue()

```
template<typename T>
Optional<T>::Optional() :
    m_value(), m_hasValue(false) {
}
```

```
template<typename T>
Optional<T>::Optional(T value) :
    m_value(std::move(value)),
    m_hasValue(false) {
}
```

```
template<typename T>
bool Optional<T>::hasValue() const {
    return m_hasValue;
}
```

```
template<typename T>
T &Optional<T>::getValue() {
    return m_value;
}
```

```
template<typename T>
const T &Optional<T>::getValue() const {
    return m_value;
}
```

Použitie šablóny

- Kompilátor pri každom použití šablóny vytvorí nový typ substitúciou typových parametrov (T) skutočnými typmi (`int`, `double`)
- Vzniknuté typy sú **samostatné** a **rozdielne**
 - V príklade tak vzniknú nové triedy s pomyselnými názvami `OptionalInt` a `OptionalDouble`

```
void optionallyPrint(Optional<double> optd) {  
    if (optd.hasValue()) {  
        const double val = optd.getValue();  
        std::cout << val*val << "\n";  
    }  
}  
  
int main() {  
    Optional<int> opti(10);  
    Optional<double> optd(1.41);  
    optionallyPrint(optd);  
}
```

Použitie šablóny

- Vzniknuté typy sú **samostatné** a **rozdielne**

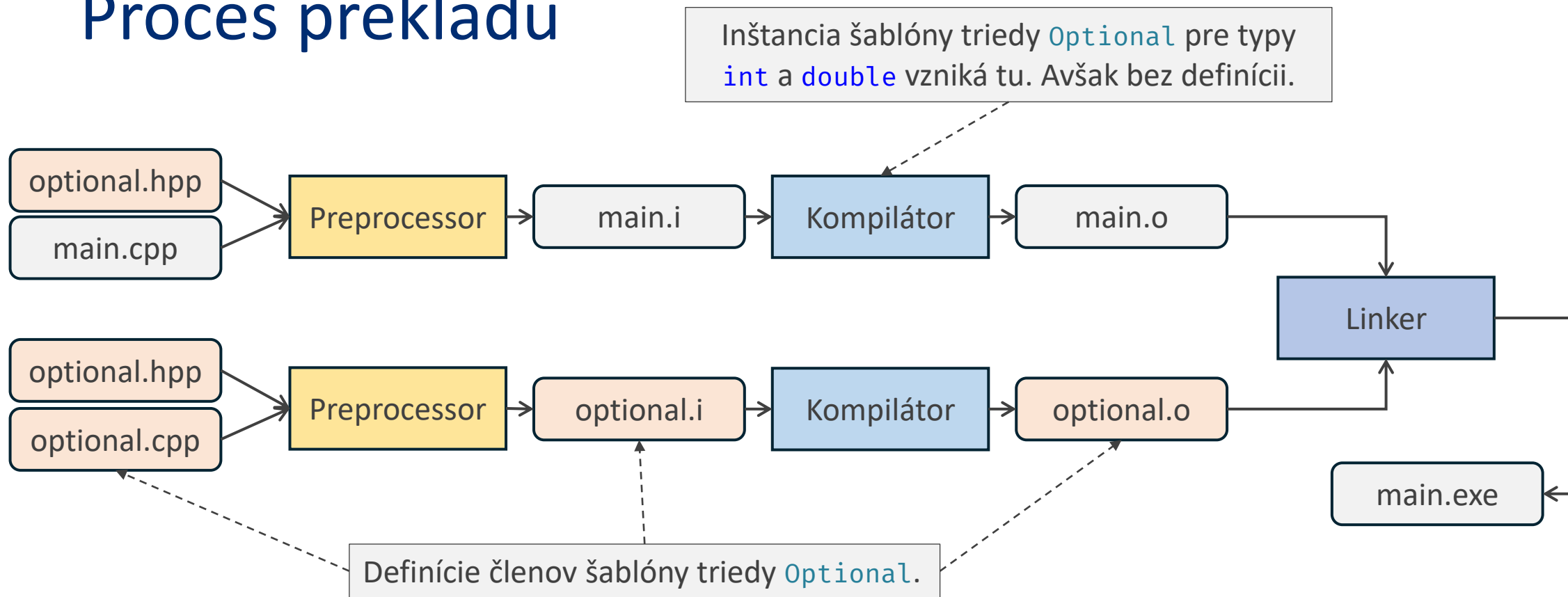
```
void optionallyPrint(Optional<double> optd) {  
    if (optd.hasValue()) {  
        const double val = optd.getValue();  
        std::cout << val*val << "\n";  
    }  
}  
  
int main() {  
    Optional<int> opti(10);  
    Optional<double> optd(1.41);  
    optionallyPrint(opti);  
}
```

optional.cpp: In function 'int main()':

optional.cpp:54:18: **error**: could not convert 'opti' from 'Optional<int>' to 'Optional<double>'

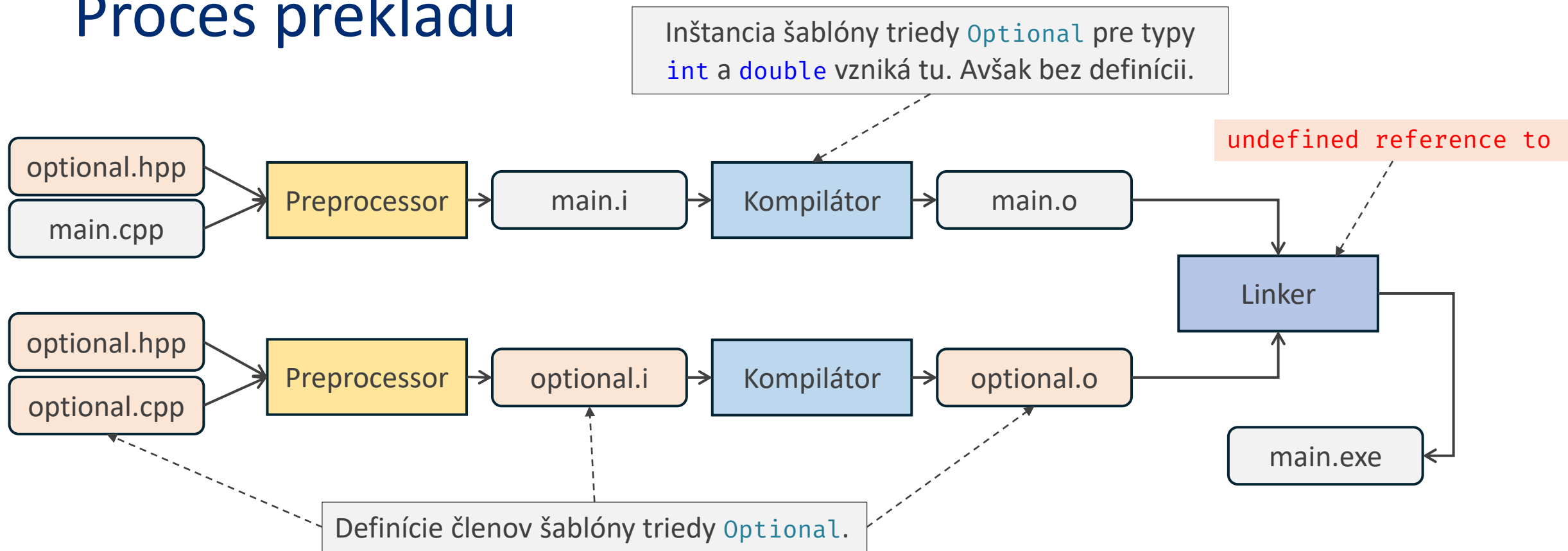
```
54 |    optionallyPrint(opti);  
    |                      ^~~~  
    |                      |  
    |                      Optional<int>
```

Proces prekladu



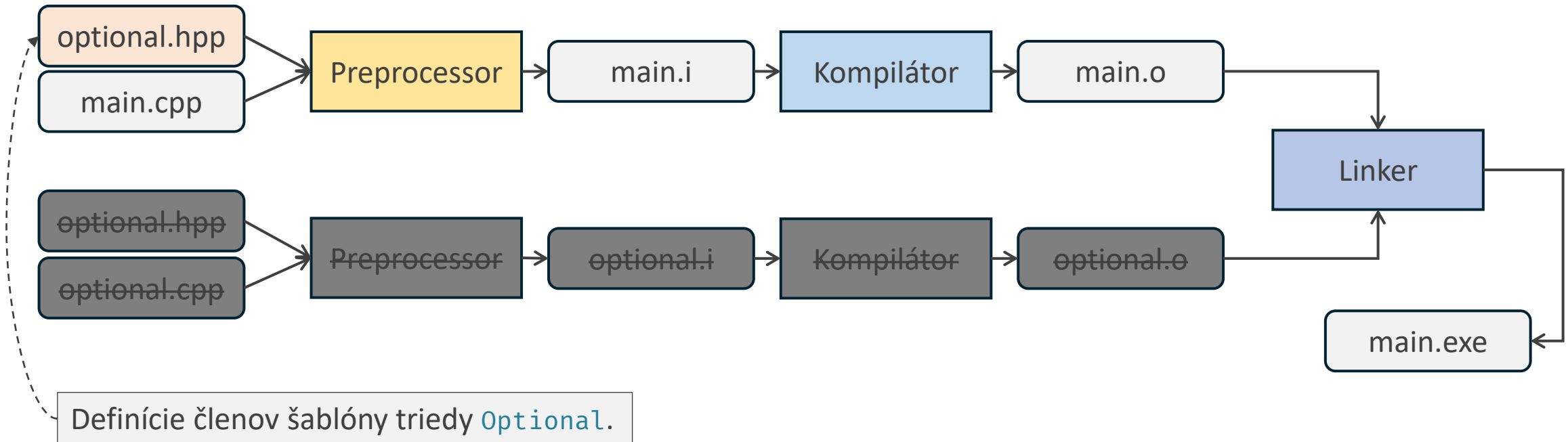
Uvažujme so štandardným rozdelením deklarácií a definícií do `hpp` a `cpp` súborov a zohľadnime fakt, že za vytvorenie šablóny je zodpovedný kompilátor.

Proces prekladu



Uvažujme so štandardným rozdelením deklarácií a definícií do `hpp` a `cpp` súborov a zohľadnime fakt, že za vytvorenie šablóny je zodpovedný kompilátor.

Proces prekladu



Riešenie: definície členov šablóny triedy musia byť k dispozícii všade, kde sú dostupné deklarácie. Prakticky to znamená, že musia byť súčasťou hlavičkového súboru `optional.hpp`.

Linkovanie



Inline (členské) funkcie

- Štandardne môže byť každá entita definovaná najviac jedenkrát
- Umiestnenie definícií členov šablóny do hlavičkového súboru triedy toto pravidlo *takmer isto* porušuje
- Preto kompilátor automaticky každú definíciu člena šablóny označí špecifikátorom `inline`
- Definícia `inline` (členskej) funkcie (bez ohľadu na to, či patrí šablóne) sa môže objaviť v rôznych jednotkách prekladu
- Definícia musí byť vo všetkých jednotkách prekladu totožná*
- Kompilátor zabezpečí, že sa použije práve jedna z definícií

Toto zvyčajne nie je problém, pretože sa do viacerých jednotiek prekladu dostane z jedného hlavičkového súboru. *

Inline členské funkcie

- Členské funkcie môžeme definovať priamo v definícii triedy
- V tomto prípade sa automaticky aplikuje špecifikátor `inline`, aj ak sa nejedná o šablónu triedy
- Pri definícii (členskej) funkcie (ktorá nepatrí šablóne) v hlavičkovom súbore je nutné špecifikovať `inline` explicitne

```
template<typename T>
class Optional {
public:
    Optional() : m_hasValue(false) {}
    Optional(T value) :
        m_hasValue(true), m_value(std::move(value)) {}

    bool hasValue() const {
        return m_hasValue;
    }

    T &getValue() {
        return m_value;
    }

    const T &getValue() const {
        return m_value;
    }

private:
    T m_value;
    bool m_hasValue;
};
```

Inline členské funkcie

- Členské funkcie môžeme definovať priamo v definícii triedy
- V tomto prípade sa automaticky aplikuje špecifikátor `inline`, aj ak sa nejedná o šablónu triedy
- Pri definícii (členskej) funkcie (ktorá nepatrí šablóne) v hlavičkovom súbore je nutné špecifikovať `inline` explicitne

```
class Circle {  
public:  
    Circle(Vec2 center, float radius) :  
        m_center(center), m_radius(radius) {  
    }  
  
    Vec2 getCenter() const;  
  
    float getRadius() const {  
        return m_radius;  
    }  
  
private:  
    Vec2 m_center;  
    float m_radius;  
};  
  
inline Vec2 Circle::getCenter() const {  
    return m_center;  
}
```

Inlinovanie funkcií

- Kľúčové slovo `inline`, ktoré **potláča pravidlo jednej definície**, sa často spomína v súvislosti s pojmom inlinovania funkcií
- Ak má kompilátor v mieste volania funkcie **k dispozícii jej definíciu**, môže sa rozhodnúť namiesto zavolania funkcie vložiť kód (inštrukcie) tejto funkcie priamo na miesto volania a vyhnúť sa tak vytváraniu rámca na zásobníku
- To, že je funkcia označená špecifikátorom `inline`, môže mierne ovplyvniť kompilátor pri rozhodovaní, **nehrá však kľúčovú úlohu**
- Táto technika sa používa pri pokročilej optimalizácii kódu, pri bežnom programovaní ju *zväčša* nemusíme zohľadňovať

```

void printCircle(const Circle &c) {
    std::cout << "Circle with radius " << c.getRadius() << "\n";
}
int main() {
    Circle c({0,0}, 1);
    printCircle(c);
}

```

```

"printCircle(Circle const&)":
    push    rbx
    mov     edx, 19
    mov     rbx, rdi
    mov     esi, OFFSET FLAT:.LC0
    mov     edi, OFFSET FLAT:"std::cout"
    call    "operator<< ..."
    mov     edi, OFFSET FLAT:"std::cout"
    pxor    xmm0, xmm0
    cvtss2sd xmm0, DWORD PTR [rbx+8]
    call    "operator<< ..."
    mov     edx, 1

```

Definícia `getRadius` je k dispozícii.

```

"printCircle(Circle const&)":
    push    rbx
    mov     edx, 19
    mov     esi, OFFSET FLAT:.LC0
    mov     rbx, rdi
    mov     edi, OFFSET FLAT:"std::cout"
    call    "operator<< ..."
    mov     rdi, rbx
    call    "Circle::getRadius() const"
    mov     edi, OFFSET FLAT:"std::cout"
    cvtss2sd xmm0, xmm0
    call    "operator<< ..."
    mov     edx, 1

```

Definícia `getRadius` nie je k dispozícii.

Inlinovanie funkcií

- Definovanie `inline` funkcií je **zdanlivo výhodné** – programátora by mohlo zvädzať definovať všetky funkcie v hlavičkovom súbore
- Takýto prístup však potláča výhody **oddelenej kompilácie**
 - Redukuje možnú paralelizáciu
 - Zvyšuje čas prekladu, keďže definície musia byť kompilované v každej jednotke prekladu
 - V reakcii na zmenu kódu (v hlavičkovom) súbore vyvoláva opätovnú kompiláciu veľkého množstva súborov
- Ako `inline` tak definujeme iba funkcie, u ktorých sme presvedčený alebo – ešte lepšie – máme odmeraný pozitívny vplyv na sledovaný ukazovateľ

Inline premenné

- Statické členské premenné deklarujeme v tele definície triedy a definujeme v jednom zdrojovom súbore (platí pre ne pravidlo jednej definície)
- Pre celočíselné konštanty platí výnimka – ich hodnotu môžeme nastaviť pri deklarácii a používať ju ako konštantný výraz bez definície (ak by sme potrebovali pracovať s jej adresou, bola by potrebná definícia)

display.hpp

```
class Display {  
public:  
    // ...  
  
private:  
    static const int N_ROWS = 8;  
  
private:  
    DisplayRow *m_rows[N_ROWS];  
};
```

display.cpp

```
int Display::N_ROWS = 8;
```

Inline premenné

- Statickú členskú premennú definujeme v zdrojovom súbore

display.hpp

```
class Display {  
public:  
    // ...  
  
private:  
    static const int N_ROWS = 8;  
    static std::string s_prefix;  
  
private:  
    DisplayRow *m_rows[N_ROWS];  
};
```

display.cpp

```
int Display::N_ROWS = 8;  
std::string Display::s_prefix("GTL87");
```

Inline premenné

- Statickú členskú premennú definujeme v zdrojovom súbore
- (od C++17) `inline` špecifikátor nám umožní definovať ju priamo v definícii triedy
- Rovnaké pravidlá platia pre globálne premenné
- Kompilátor zabezpečí, že sa použije jedna z definícií (premenná bude v pamäti iba raz)

display.hpp

```
class Display {  
public:  
    // ...  
  
private:  
    inline static const int N_ROWS = 8;  
    inline static std::string s_name{"GLT87"};  
  
private:  
    DisplayRow *m_rows[N_ROWS];  
};
```

display.cpp

Väzba (angl. linkage)



Väzba

- Každý symbol (funkcia, premenná, trieda) má vlastnosť, ktorú môžeme preložiť ako väzba (angl. linkage)
- Väzba ovplyvňuje, či je daný symbol viditeľný v iných jednotkách prekladu
- Jazyk C++ rozlišuje nasledovné väzby:
 - **Vnútorá (internal)** – symbol je viditeľný (môže byť (re)deklarovaný) iba v jednotke prekladu, v ktorej je definovaný
 - **vonkajšia (external)** – symbol je viditeľný (môže byť (re)deklarovaný) aj v iných jednotkách prekladu
 - **Žiadna (no)** – symboly so žiadnou väzbou (môžu byť (re)deklarované iba v rozsahu svojej platnosti)

Vnútna väzba

- Väčšina entít (triedy, funkcie), s ktorými bežne pracujeme má **vonkajšiu väzbu**
- **Vnútnú väzbu** môžeme vynútiť kľúčovým slovom `static` alebo umiestnením symbolu do anonymného menného priestoru
- Vnútna väzba má niekoľko výhod:
 - Ide o analógiu `private` modifikátora prístupu na úrovni linkovania – ukrývanie implementačných detailov
 - Dáva kompilátoru viac priestoru na optimalizácie
 - Zmenšuje šancu na potenciálny konflikt symbolov
 - Zmenšuje veľkosť binárnych súborov

```

void printCircle(const Circle &c) {
    std::cout
        << "Circle with radius „
        << c.getRadius() << "\n";
}
int main() {
    Circle c({0,0}, 1);
    printCircle(c);
}

```

Funkcia `printCircle` nie je po optimalizácii inlinovaním nikde vo funkcii `main` zavolaná.

Napriek tomu kompilátor vygeneroval do spustiteľného binárneho súboru jej definíciu.

Niektó v inej jednotke prekladu by ju totiž mohol zadeklarovať a zavolať, pretože má vonkajšiu väzbu (externé likovanie).

```

"printCircle(Circle const&)":
    push    rbx
    mov     edx, 19
    mov     rbx, rdi
    # ...
"main":
    sub     rsp, 8
    mov     edx, 19
    mov     esi, OFFSET FLAT:.LC0
    mov     edi, OFFSET FLAT:"std::cout"
    call    "operator<< ... "
    movsd   xmm0, QWORD PTR .LC2[rip]
    mov     edi, OFFSET FLAT:"std::cout"
    call    "operator<< ... "
    mov     edx, 1
    mov     esi, OFFSET FLAT:.LC1
    mov     rdi, rax
    call    "operator<< ... "
    xor     eax, eax
    add     rsp, 8
    ret

```

```
static void printCircle(const Circle &c) {
    std::cout
        << "Circle with radius „
        << c.getRadius() << "\n";
}
int main() {
    Circle c({0,0}, 1);
    printCircle(c);
}
```

Po zmene jej väzby na vnútornú zmizne jej definícia z binárneho spustiteľného súboru.

Kompilátor to môže spraviť, pretože vie, že nikde inde v danej jednotke prekladu nie je použitá a má vnútornú väzbu (interné linkovanie).

```
"main":
    sub     rsp, 8
    mov     edx, 19
    mov     esi, OFFSET FLAT:.LC0
    mov     edi, OFFSET FLAT:"std::cout"
    call    "operator<< ... "
    movsd   xmm0, QWORD PTR .LC2[rip]
    mov     edi, OFFSET FLAT:"std::cout"
    call    "operator<< ... "
    mov     edx, 1
    mov     esi, OFFSET FLAT:.LC1
    mov     rdi, rax
    call    "operator<< ... "
    xor     eax, eax
    add     rsp, 8
    ret
```


Vnútoraná väzba

- Na nastavenie vnútornej väzby môžeme (od C++11) tiež použiť anonymný menný priestor
- Jeho použitie je preferované
- Umožňuje jednoducho definovať triedu s vnútornou väzbou
- Vnútoraná väzba má zmysel pri všetkých *pomocných* entitách, ktoré nepotrebujeme v iných jednotkách prekladu

```
namespace {  
    void printCircle(const Circle &c) {  
        std::cout  
            << "Circle with radius „  
            << c.getRadius() << "\\n";  
    }  
}  
  
int main() {  
    Circle c({0,0}, 1);  
    printCircle(c);  
}
```

Šablóna funkcie a špecializácia



Šablóna funkcie

- Okrem šablóny tried je veľmi časté použitie šablón funkcie
- Ich definícia a vlastnosti sa riadia rovnakými pravidlami ako šablóny tried
- Typicky ich definujeme v hlavičkových súboroch – alebo zdrojových súboroch, ak sú použité iba v danej jednotke prekladu

```
template<typename T, typename U>
void my_swap(T &a, U &b) {
    T tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int a = 10;
    int b = 20;
    std::cout << a << " " << b << "\n";
    my_swap<int, int>(a, b);
    std::cout << a << " " << b << "\n";
}
```

Dedukcia typov

- Pri volaní šablóny funkcie nie je potrebné explicitne uvádzať hodnoty typových parametrov
- Kompilátor ich vydedukuje podľa typov výrazov
- Následne vytvorí inštanciu šablóny pre dané typy

```
template<typename T, typename U>
void my_swap(T &a, U &b) {
    T tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int a = 10;
    int b = 20;
    std::cout << a << " " << b << "\n";
    my_swap(a, b);
    std::cout << a << " " << b << "\n";
}
```

(úplná) Špecializácia šablón

- Šablónu môžeme špecializovať pre vybraný typ poskytnutím definície pre tento typ
- Pri volaní funkcie s daným typom sa vyberie špecifickejšia (špecializovaná) implementácia
- Úplná špecializácia však už nie je šablóna, preto, ak definíciu umiestnime do hlavičkového súboru, musím použiť špecifikátor `inline`

```
template<typename T, typename U>
void my_swap(T &a, U &b) {
    T tmp = a;
    a = b;
    b = tmp;
}

template<>
inline void my_swap<int, int>(int &a, int &b) {
    a = b ^ a;
    b = a ^ b;
    a = b ^ a;
}
```

(čiastočná) Špecializácia šablón

- Šablónu triedy ((od C++14) aj premennej) je možné čiastočne špecializovať
- Špecializovať môžeme typ jedného parametra (príklad)
- Môžeme tiež poskytnúť hodnotu niektorých z parametrov
- Kompilátor vyberie najšpecifickejšiu definíciu
- Často sa využíva v pokročilom generickom programovaní

```
template<class T>
struct Printer {
    void print(std::ostream &ost, T val) const {
        ost << "value " << val << "\n";
    }
};

template<class T>
struct Printer<T*> {
    void print(std::ostream &ost, T *val) const {
        ost << "address " << val
            << " of value" << *val << "\n";
    }
};

int main() {
    int x = 10;
    Printer<int> pi;
    Printer<int*> ppi;
    pi.print(std::cout, x);
    ppi.print(std::cout, &x);
}
```

Konštantné parametre šablóny

- Okrem typových parametrov môže byť šablóna parametrizovaná aj konštantnými ((do C++26) netypovými) **parametrami**
- Parametrom môže byť hodnota (okrem iných):
 - celočíselného typu
 - typu ukazovateľ
 - (od C++20) reálneho typu

```
template<typename T, int Size>
class Array {
public:
    T &operator[](int i) {
        return m_data[i];
    }

    const T &operator[](int i) const {
        return m_data[i];
    }

    int size() const {
        return Size;
    }

private:
    T m_data[Size];
};
```

Štandardná knižnica ich používa na v definícii šablóny triedy `std::array`.

Duck typing



Duck typing

- Šablónu funkcie `max` môžeme zavolať s ľubovoľným typom
- Ak je pre daný typ definovaný `operator<`, kód bude fungovať
- Ak nie, dostaneme chybu pri preklade

```
main.cpp:30:20:   required from here
  30 |         Degrees m = max(d1, d2);
      |                        ~~~^~~~~~
utils.hpp:21:14: error: no match for 'operator<'
(operand types are 'const Degrees' and 'const Degrees')
  21 |         return a > b ? a : b;
```

```
max :: (Ord a) => a -> a -> a
max x y = if x < y then y else x
```

```
template<typename T>
T max(const T &a, const T &b) {
    return a < b ? b : a;
}
```

```
struct Degrees {
    float value;
};

int main() {
    Degrees d1{20};
    Degrees d2{30};
    Degrees m = max(d1, d2);
    std::cout << m.value << "\n";
}
```

Duck typing

```
inline bool operator<(const Degrees &a, const Degrees &b) {  
    return a.value < b.value;  
}
```

Po definovaní operátora bude kód fungovať.

Rovnako bude kód fungovať pre *čokoľvek*, čo je porovnateľné
operátorom<.

Definícia operátora je *rozumný* kandidát na `inline` funkciu.

```
template<typename T>  
T max(const T &a, const T &b) {  
    return a < b ? b : a;  
}
```

```
struct Degrees {  
    float value;  
};  
  
int main() {  
    Degrees d1{20};  
    Degrees d2{30};  
    Degrees m = max(d1, d2);  
    std::cout << m.value << "\n";  
}
```

Príklad

- Každá hrateľná rasa má hlavnú budovu a robotníka
- Robotník nosí do hlavnej budovy suroviny (zlato)
- Hlavná budova si potrebuje uchovávať zoznam robotníkov, ktorí aktuálne nosia suroviny
- Hlavná budova vie **vyrábať robotníkov**



```

template<typename TWorker>
class MainBuilding {
public:
    const std::vector<TWorker*> &getWorkers() const {
        return m_workers;
    }

    void takeGold(TWorker *worker) {
        m_gold += worker->releaseGold();
    }

    TWorker *createWorker() {
        return new TWorker();
    }

    // ...

private:
    std::vector<TWorker*> m_workers;
    int m_gold;
};

```

Šablóny nám umožňujú vytvárať inštancie typového parametra, pretože máme k dispozícii skutočný typ – v kontraste s mechanizmom type erasure v jazyku Java.

```

class Worker {
public:
    int releaseGold() {
        const int ret = m_gold;
        m_gold = 0;
        return ret;
    }

private:
    int m_gold;
};

class Peon : public Worker {
public:
    void hideToBurrow();
};

class Peasant : public Worker {
public:
    void activateMilitia();
};

```

Hlavná budova predpokladá, že za typový parameter `TWorker` bude dosadený *niekto*, kto vie priniesť zlato.

```
int main() {
    TownHall townHall;
    GreatHall greatHall;
    Peasant *peasant = townHall.createWorker();
    Peon *peon = greatHall.createWorker();
}
```

Šablóny môžeme spojiť s mechanizmom dedičnosti. Potomkovia často špecifikujú typový parameter svojho predka.

Uvedený príklad by bolo možné riešiť aj s použitím dedičnosti a polymorfizmu – bez šablón.

Použitím šablóny sa však zbavíme jednej vrstvy abstrakcie a pretypovaní a poskytneme kompilátoru viac priestoru na optimalizácie.

```
class TownHall : public MainBuilding<Peasant> {
public:
    void callMilitia() {
        for (Peasant *p : this->getWorkers()) {
            p->activateMilitia();
        }
    }
};

class GreatHall : public MainBuilding<Peon> {
    // ...
};
```

Koncepty

- Výhodou mechanizmu *duck typing* je, že môžeme veľmi jednoducho písať generický kód
- Problém je, ak by sme chceli obmedziť doménu niektorého z parametrov
- Obzvlášť v minulosti spôsobovala typová nekompatibility dlhé a ťažšie čitateľné hlásenia chýb
- Tento problém stále pretrváva pri práci s rozsiahlym generickým kódom, kompilátory sa však značne zlepšili v čitateľnosti chybových hlásení
- (od C++20) Na bližšie špecifikovanie vlastností typových parametrov môžeme použiť koncepty*

Rovnaký pojem používame pre koncepty jazyka C++ a na všeobecné označenie obmedzení typového parametra. *

Koncepty

- Koncept nám umožní špecifikovať aké operácie musí typový parameter podporovať
- Tiež môžeme užšie špecifikovať jeho typ (celé číslo, trieda, ukazovateľ, ...)
- Štandardná knižnica definuje niekoľko konceptov v hlavičke `<concepts>`
- Koncept zapisujeme namiesto kľúčového slova `typename`

```
template<std::totally_ordered T>
T max(const T &a, const T &b) {
    return a < b ? b : a;
}
```

Koncept `std::totally_ordered` definuje, že typ `T` musí byť porovnateľný relačnými operátormi.

```
main.cpp: In function 'int main()':
main.cpp:33:18: error: no matching function for call to 'max(Dummy, Dummy)'
   33 |         Dummy m = max(Dummy{}, Dummy{});
      |                     ~~~~~^~~~~~
main.cpp:33:18: note: there is 1 candidate
utils.hpp:21:3: note: constraints not satisfied
```

Dôsledky použitia šablón



Dôsledky použitia šablón

- Základné správanie vieme reprodukovať použitím polymorfizmu a dedičnosti
- Pri použití šablón vytvárame nový typ „na mieru“ pre každú sadu typových parametrov
- Kompilátor tak môže optimalizovať kód pre konkrétne typy
- Rozsiahle použitie šablón môže značne zvýšiť čas prekladu projektu a zväčšiť veľkosť binárnych súborov
- Mnohé knižnice využívajúce šablóny sú definované iba v hlavičkových súboroch – sú veľmi jednoduché na integráciu – napríklad knižnice [Boost](#)
- Šablóny umožňujú pokročilé generické programovanie a [metaprogramovanie](#)

Ďakujem za pozornosť

Priestor na otázky

